

Operational Dashboard – Work details

First assess the

- Need (is this measure valuable?)
- Availability (do we have the required data?)
- Feasibility (can the available data be rendered?)

EPIC 1 Reliability core

Epic goal. *Give leadership an at-a-glance answer to: are we meeting our reliability commitments, and are we getting better or worse?*

EPIC 2 Service health matrix

Epic goal. *Give leadership the answer: which specific services are unhealthy right now, and how? This is the layer the Epic 1 tiles drill into.*

EPIC 3 Delivery velocity (DORA)

Epic goal. *Give leadership a credible, industry-standard answer to: how fast and how safely do we ship software?*

EPIC 4 Experience and depth

Epic goal. *Give leadership the user-experience signals and error detail to answer: is the real experience fast and reliable, and what is the error picture? These enrich the team view; they are not leadership headlines.*

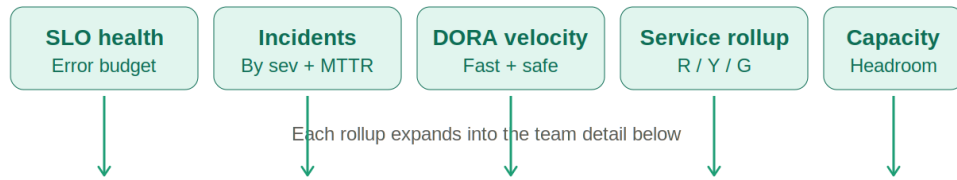
EPIC 5 Cost and efficiency

Epic goal. *Give leadership an efficiency view: are we spending sensibly on the systems and tooling that run the business? Built last; one story is gated on a licensing decision.*

Draft structure

Leadership view

Trends and rollups — "are we healthy and improving?"



Team view

Current state and detail — "what's wrong and where?"

Service health matrix

Throughput, error rate
p50 / p95 / p99 latency
Status per service

Firing monitors

Live alert list + severity
Slowest endpoints
Top error producers

RUM experience

Core Web Vitals (p75)
Frontend errors by app
Synthetic journey results

Logs & errors

Error volume + spikes
Top error patterns
Correlated to deploys

Dependency map

Service-to-service blast radius for incident triage

Summary-tile pattern: one dashboard, two altitudes, no duplicated data

Leadership reads trends; teams read current state — same underlying objects

EPIC 1 Reliability core

Epic goal. *Give leadership an at-a-glance answer to: are we meeting our reliability commitments, and are we getting better or worse?*

US-1.1 SLO attainment and error budget tile

Terms. An SLO (Service Level Objective) is a reliability target a team commits to, e.g. “99.9% availability.” The error budget is the allowed amount of failure before the target is breached — if the target is 99.9%, the budget is the remaining 0.1% (about 43 minutes of downtime per month).

As a technology leader, I want to see each critical SLO’s current attainment and how much error budget is left, so that I can tell at a glance whether we are keeping our reliability promises.

Acceptance criteria

- ✓ Given a list of critical SLOs, when the tile loads, then each SLO shows its name, current attainment vs. target, and error budget remaining as a percentage.
- ✓ Given an SLO’s remaining budget, when it is healthy / low (e.g. under 25%) / exhausted, then the row is colored green / yellow / red respectively.
- ✓ Given an SLO with no data, when the tile loads, then it shows “no data” rather than a misleading 100%.
- ✓ The user can change the time window and the list of critical SLOs is read from editable configuration, not hardcoded.

Example

Checkout availability — Target 99.9% — Currently 99.95% — Error budget remaining: 68% (green)

Engineering notes

- Error budget is normally measured over a rolling 30 days; make the window configurable.

US-1.2 Active incidents by severity tile

Terms. An incident is a logged event where engineers formally respond to a problem. Severity (SEV-1 most severe through SEV-5 minor) ranks how bad it is.

As a technology leader, I want to see how many incidents are active right now, grouped by severity and with how long each has been open, so that I know the current operational pressure without asking anyone.

Acceptance criteria

- ✓ Given active (unresolved) incidents, when the tile loads, then it shows a count per severity level.
- ✓ Given each active incident, when displayed, then it shows how long it has been open (now minus start time), most severe first.

- ✓ Given zero active incidents, when the tile loads, then it shows a clear “All clear — no active incidents” state, not an empty box.

Example

Active incidents: 1 SEV-1 (open 47 min), 2 SEV-3 (open 2h 10m, 35 min)

US-1.3 Mean time to resolve (MTTR) trend tile

Terms. MTTR (Mean Time To Resolve) is the average time incidents take to fix, from start to resolution — a measure of how fast we recover.

As a technology leader, I want to see the average incident resolution time as a trend over time, so that I can tell whether our recovery speed is improving or degrading.

Acceptance criteria

- ✓ Given resolved incidents in a chosen period, when the tile loads, then it shows the average resolution time (resolved time minus start time).
- ✓ Given multiple periods, when displayed, then the value is shown as a trend so direction of change is visible.
- ✓ Given a choice, when the user filters by severity, then the average recalculates for that severity only.

Example

Mean time to resolve (last 90 days): 3h 12m — trending down from 4h 5m last quarter

Engineering notes

- One or two very long incidents can distort an average; showing the median alongside the mean is a useful refinement.

US-1.4 Monitor and alert rollup tile

Terms. A monitor is an automated rule that watches a metric and changes state when a threshold is crossed. States are OK, Warn, Alert, and No Data (Datadog isn't receiving the expected signal).

As a technology leader, I want to see how many monitors are currently in each state, so that I get a one-glance sense of how much is firing across the estate.

Acceptance criteria

- ✓ Given all monitors, when the tile loads, then it shows a count for each state (OK, Warn, Alert, No Data).
- ✓ Given the “No Data” count, when displayed, then it is shown explicitly (not hidden), because it often means a silently broken monitor.
- ✓ Given a click on the tile, when activated, then it drills into the firing-monitors list (US-2.4).

Example

4 in Alert, 7 in Warn, 142 OK, 3 No Data

Stories common to Epic 1 (apply to every tile above)

US-1.X Tile display states and refresh

As a platform developer, I want every reliability tile to handle loading, data, and no-data states and refresh on a schedule, so that leadership never sees a blank or a misleading zero, and we don't overload Datadog.

Acceptance criteria

- ✓ Given any tile, when data is loading, then a loading state is shown; when no data exists, then a clear no-data state is shown.
- ✓ Given the dashboard is open, when time passes, then tiles refresh on a schedule of every 1–5 minutes (not continuously).
- ✓ Given any “critical” list (SLOs, services, monitors), when configured, then it is read from editable configuration.

EPIC 2 Service health matrix

Epic goal. Give engineers the detail to answer: which specific services are unhealthy right now, and how? This is the layer the Epic 1 tiles drill into.

US-2.1 Per-service health matrix

Terms. Throughput is requests handled per second. Error rate is the percentage of requests that failed. Latency at p50/p95/p99 shows how long requests take at the median, the slow 5%, and the worst 1% — three numbers because an average can look fine while a real slice of users suffers.

As an engineer, I want a table with one row per service showing its key health numbers side by side, so that I can scan the whole estate and immediately spot the problem service.

Acceptance criteria

- ✓ Given all in-scope services, when the matrix loads, then each row shows throughput, error rate, p50/p95/p99 latency, and a green/yellow/red status.
- ✓ Given any column, when the engineer clicks its header, then the table sorts by that column (e.g. worst error rate or p99 first).
- ✓ Given a service's thresholds, when status is computed, then the green/yellow/red bands are configurable per service.
- ✓ The user can change the time window.

Example

checkout-api — 1,240 req/s — error rate 0.3% — p50 45ms / p95 210ms / p99 890ms — status: green

Reading it: the p99 of 890ms shows 1% of requests take nearly a second — which an average would have hidden.

US-2.2 Slowest endpoints drill-down

Terms. An endpoint is one operation a service exposes, e.g. "POST /checkout." A service can be healthy overall while one endpoint is slow.

As an engineer, I want to click into a service and see its endpoints ranked slowest-first, so that I can pinpoint exactly which operation is dragging performance.

Acceptance criteria

- ✓ Given a service in the matrix, when the engineer clicks it, then its endpoints are listed ranked by latency, slowest first.
- ✓ Given each endpoint, when listed, then its latency (e.g. p95) is shown next to it.

Example

Within checkout-api: POST /apply-coupon p95 1,400ms (slowest), POST /checkout p95 230ms, GET /cart p95 60ms

US-2.3 Top error-producing services view

As an engineer, I want a ranked list of the services producing the most errors right now, so that I can go straight to the worst offenders without sorting the full matrix.

Acceptance criteria

- ✓ Given all services, when the view loads, then it lists those producing the most errors, highest first, with error counts.
- ✓ The user can change the time window (e.g. last hour vs last day).

Example

Most errors (last hour): payment-service 412, search-service 88, user-auth 12

US-2.4 Firing monitors list (Epic 1 drill-down)

As an engineer, I want to land on a filtered list of currently-firing monitors when I click the monitor rollup, so that I can move from “something is firing” to “here is exactly what” in one click.

Acceptance criteria

- ✓ Given the monitor rollup tile (US-1.4), when clicked, then this list opens showing all monitors in Alert or Warn state.
- ✓ Given each firing monitor, when listed, then it shows the monitor name, severity, related service, and how long it has been firing.
- ✓ Given the list, when the engineer filters by service or state, then the list updates accordingly.
- ✓ Given a monitor, when clicked, then the engineer can open its underlying detail in Datadog.

Example

“checkout error rate > 5%” — Alert — checkout-api — firing 12 min

“search latency high” — Warn — search-service — firing 1h 4m

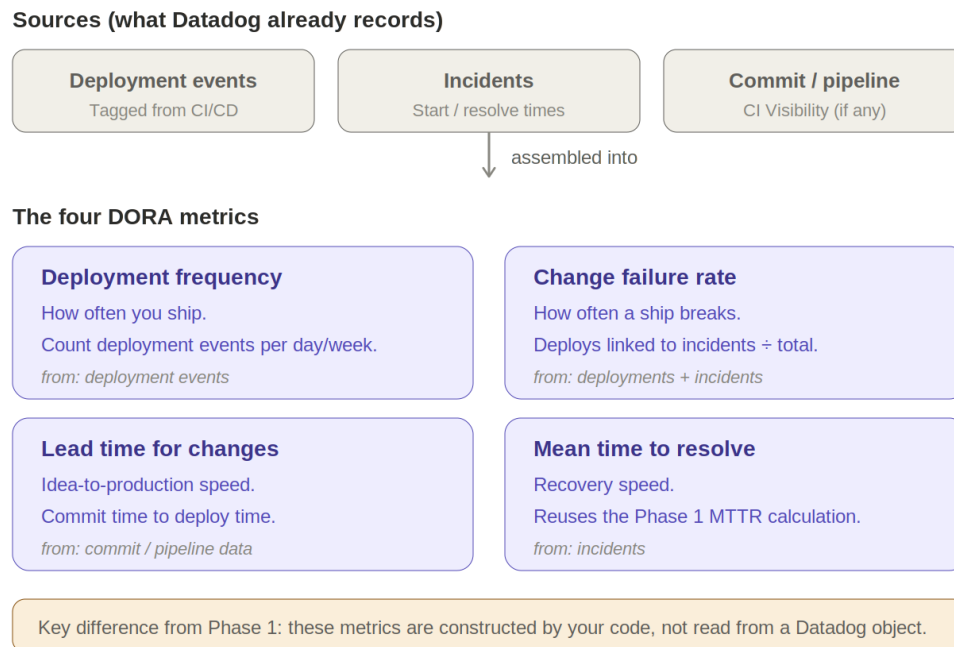
Engineering notes

- This whole epic depends on Epic 0 service tagging being clean; with inconsistent tags the matrix will be noisy.

EPIC 3 Delivery velocity (DORA)

Epic goal. Give leadership a credible, industry-standard answer to: how fast and how safely do we ship software?

What DORA is. DORA (DevOps Research and Assessment) is a standard set of four delivery-performance metrics. Unlike SLOs and incidents, DORA metrics are not ready-made Datadog objects — the team assembles them from deployment events, incidents, and (for one metric) CI/CD pipeline data. CI/CD is the automated system that builds and ships code. The diagram shows which source feeds which metric.



US-3.1 Deployment frequency tile

Terms. A deployment event is a record (usually auto-sent from CI/CD) that a service version went live. Frequent small releases are generally safer than rare large ones.

As a technology leader, I want to see how often we deploy to production, as a trend and broken down by service, so that I can judge our delivery cadence and spot teams that rarely ship.

Acceptance criteria

- ✓ Given deployment events, when the tile loads, then it shows deployments per day or per week as a trend.
- ✓ Given the data, when grouped, then the user can break it down by service.
- ✓ Given a team that emits no deployment events, when displayed, then it shows zero and the gap is flagged rather than hidden.

Example

Deployment frequency (last 30 days): 4.2/day across all services — up from 3.1 last month

US-3.2 Change failure rate tile

As a technology leader, I want to see what percentage of deployments caused a problem, so that I can tell how safely we are shipping, not just how often.

Acceptance criteria

- ✓ Given deployments and incidents, when the tile loads, then it shows the percentage of deployments linked to an incident, as a trend.
- ✓ Given the linking rule (e.g. an incident starting within N minutes of a deploy to the same service), when configured, then the window N is editable.
- ✓ Given the displayed number, when a leader asks how “failure” is judged, then the linking method is visible/documented in the UI.

Example

Change failure rate (last 30 days): 6% — 3 of 50 deployments led to an incident

US-3.3 Lead time for changes tile

Terms. A commit is one saved code change; its timestamp marks when work was done. Lead time is the gap from commit to running in production.

As a technology leader, I want to see the typical time from code commit to production, as a trend, so that I can tell how quickly we turn work into delivered value.

Acceptance criteria

- ✓ Given commit and deployment timestamps, when the tile loads, then it shows the median commit-to-production time as a trend.
- ✓ Given the data source, when set up, then it draws on CI/CD pipeline data (Datadog CI Visibility if licensed, otherwise the pipeline tool).
- ✓ Median is used rather than average so a few long-running changes don’t distort it.

Example

Lead time for changes (last 30 days): median 2.5 days — trending down from 4 days

Engineering notes

- Confirm early whether CI Visibility is licensed; this is the main effort/uncertainty in this story.

US-3.4 Combined DORA velocity panel

As a technology leader, I want to see all four DORA metrics together in one panel, so that I read delivery performance as a single coherent story.

Acceptance criteria

- ✓ Given the four metrics (US-3.1, US-3.2, US-3.3, and MTTR reused from US-1.3), when the panel loads, then all four appear together.
- ✓ Given the panel, when possible, then metrics can be grouped by team or service.
- ✓ MTTR is reused from US-1.3 with no new data work — presentation only.

Example

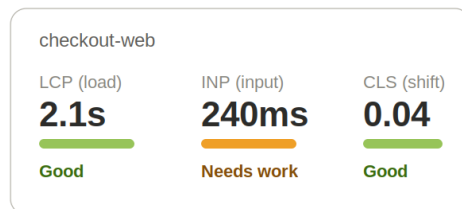
Panel: Deploy freq 4.2/day · Change failure 6% · Lead time 2.5d · MTTR 3h 12m

EPIC 4 Experience and depth

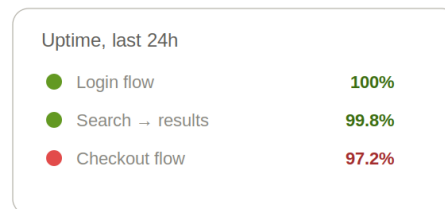
Epic goal. Give engineers the user-experience signals and error detail to answer: *is the real experience fast and reliable, and what is the error picture? These enrich the team view; they are not leadership headlines.*

Visual reference. The mockup sketches the intended shape of three Epic 4 tiles — Core Web Vitals scores, synthetic journey uptime, and error-log volume with a deploy marker.

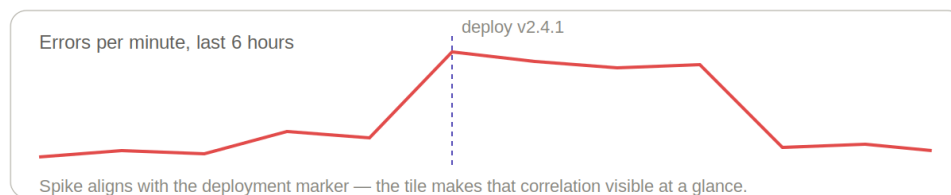
Core Web Vitals tile (per application, p75)



Synthetic journey tile (critical flows)



Error-log volume tile (with deploy markers)



US-4.1 Real user experience tile (RUM)

Terms. RUM (Real User Monitoring) is data from actual users' devices, not synthetic tests. Core Web Vitals are three standard front-end measures: LCP (load speed), INP (responsiveness to input), and CLS (visual stability). “p75” reports the experience of the slower 25% of users — the published standard for these metrics.

As an engineer, I want to see real users' Core Web Vitals per application, with good/needs-work/poor banding, so that I know what real people actually experience, not just server-side numbers.

Acceptance criteria

- ✓ Given an application, when the tile loads, then it shows LCP, INP, and CLS at p75, each banded good / needs work / poor.
- ✓ Given the tile, when displayed, then it also shows front-end error count and active session count.
- ✓ Given available data, when possible, then the user can break results down by geography and device type.

Example

checkout-web — LCP 2.1s (good) — INP 240ms (needs work) — CLS 0.04 (good) — 18 front-end errors/hr — 1,240 active sessions

US-4.2 Synthetic journey tile

Terms. A synthetic test is a scripted check that imitates a user completing a key journey (login, search, checkout), run on a schedule from outside the system — it catches breakage before a real user hits it.

As an engineer, I want to see each critical journey's recent uptime and response time with a pass/fail indicator, so that I know whether customers can actually complete key flows right now.

Acceptance criteria

- ✓ Given critical journeys, when the tile loads, then each shows recent success rate (uptime), response time, and a clear pass/fail status.
- ✓ Given the list of journeys, when configured, then which journeys count as “critical” is editable.
- ✓ Given a failing journey, when clicked, then the engineer can open the underlying synthetic result in Datadog.

Example

Login flow 100% (green) · Search→results 99.8% (green) · Checkout 97.2% (red, investigate)

US-4.3 Error-log volume with deploy markers

Terms. Logs are records services write as they run; error logs are the subset signalling something went wrong.

As an engineer, I want to see error-log volume over time with deployment markers overlaid, so that I can instantly see when an error spike started right after a release.

Acceptance criteria

- ✓ Given error logs, when the tile loads, then it shows error volume as a trend line.
- ✓ Given deployment events, when overlaid, then deploy markers appear on the same timeline so spike-to-deploy correlation is visible.
- ✓ Given the tile, when displayed, then it lists the most frequent error messages.

Example

Errors/min (last 6h): spike at 14:30 aligns with deploy v2.4.1. Top error: “payment gateway timeout” (412)

US-4.4 Service dependency map

As an engineer, I want a map of which services call which, with unhealthy ones highlighted, so that during an incident I can see the blast radius and what else is at risk.

Acceptance criteria

- ✓ Given service call data (from APM traces), when the map loads, then services appear as nodes and their calls as connections.
- ✓ Given an unhealthy service, when shown, then it and its directly dependent services are highlighted.

Example

payment-service unhealthy — map highlights checkout-api and order-service as directly dependent

EPIC 5 Cost and efficiency

Epic goal. Give leadership an efficiency view: are we spending sensibly on the systems and tooling that run the business? Built last; one story is gated on a licensing decision.

Licensing note. Datadog's own usage data is available today. True cloud-provider spend needs a separate Datadog module, Cloud Cost Management, which is not currently licensed — so US-5.2 is conditional on that purchase.

US-5.1 Datadog usage and efficiency tile (available now)

As a technology leader, I want to see our Datadog usage trends (data ingested, custom metrics, hosts), so that I can tell whether observability cost is climbing and act before it surprises me.

Acceptance criteria

- ✓ Given Datadog usage data, when the tile loads, then it shows data ingestion volume, custom metric count, and host count as trends.
- ✓ Given month-over-month change, when displayed, then notable increases are highlighted (often an unintended cost leak).
- ✓ This story needs no extra licensing and can be built whenever the epic is scheduled.

Example

Datadog usage (this month): 1.2 TB ingested, 4,300 custom metrics, 210 hosts — ingestion up 12% vs last month

US-5.2 Cloud spend tile (requires Cloud Cost Management)

As a technology leader, I want to see cloud spend over time broken down by service or team, alongside reliability and traffic, so that I can weigh what each part of the system costs against the value it delivers.

Acceptance criteria

- ✓ Given Cloud Cost Management is licensed, when the tile loads, then it shows cloud spend over time broken down by service or team.
- ✓ Given the spend data, when shown, then it sits alongside reliability and traffic context from earlier epics (cost per unit of value).
- ✓ Given the module is not yet licensed, when planning, then this story is clearly marked as blocked and not started until the module is confirmed.

Example

[Pending license] Cloud spend (this month): \$48k — checkout-service \$9k, search-service \$14k, up 8%

ROADMAP The epics at a glance

Epics are sequenced by value-to-effort. Epic 1 (reliability) leads because error budgets and incidents are ready-made Datadog objects — mostly wiring, high credibility, low effort. Epic 2 (service matrix) is the team detail the Epic 1 tiles drill into. Epic 3 (DORA) requires constructing metrics from raw events, so it follows once the reliability base is trusted. Epics 4 (experience) and 5 (cost) enrich the picture and build last. Epic 0 (foundation) gates everything and is done first.

